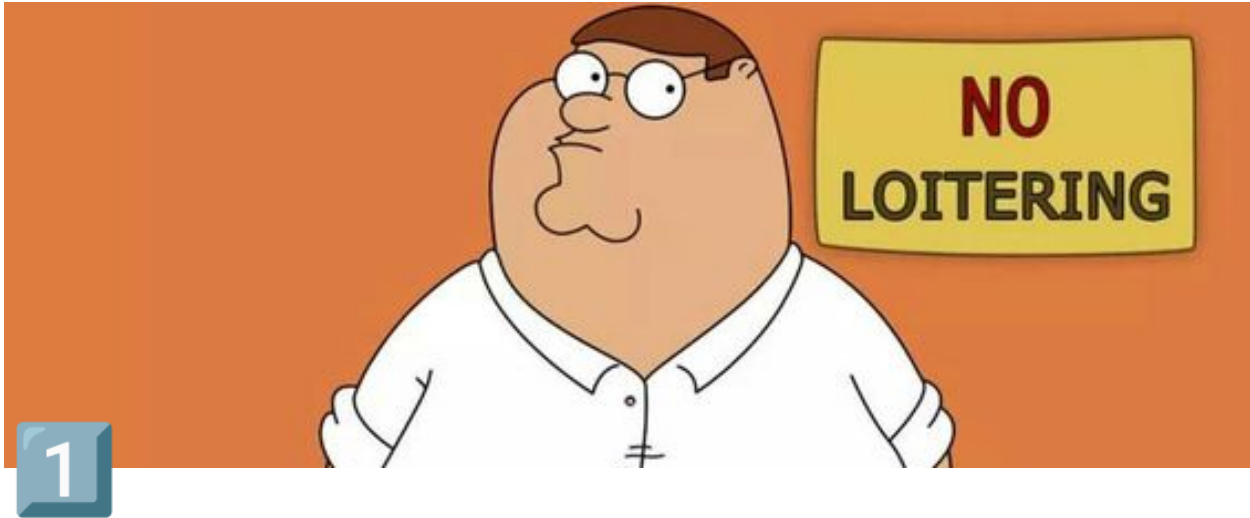
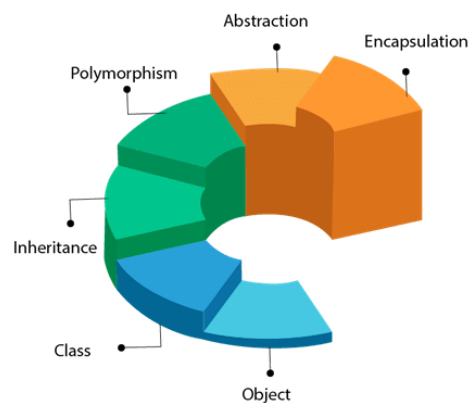




# Java: Module 1

## ▼ Object oriented programming concepts

OOPs (Object-Oriented Programming System)



**Object** means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

### ▼ Object



An Object can be defined as an instance of a class. An object contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

**Example:** A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

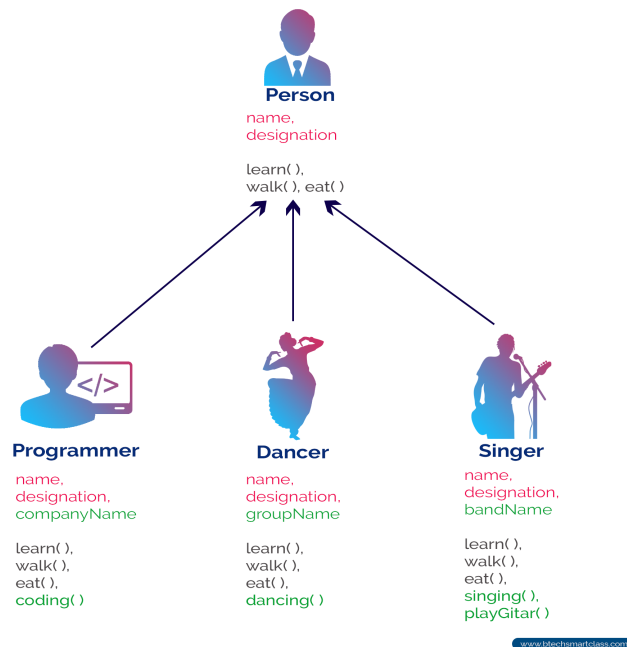
#### ▼ Class

*Collection of objects* is called class. It is a logical entity.

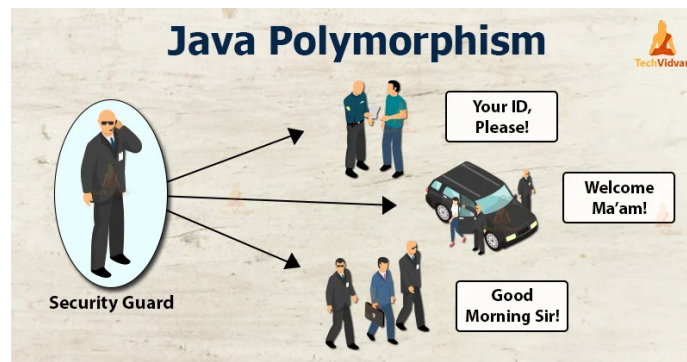
A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

#### ▼ Inheritance

***When one object acquires all the properties and behaviors of a parent object***, it is known as **inheritance**. It provides code reusability. It is used to achieve runtime polymorphism.



## ▼ Polymorphism



**If one task is performed in different ways, it is known as polymorphism.**

For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

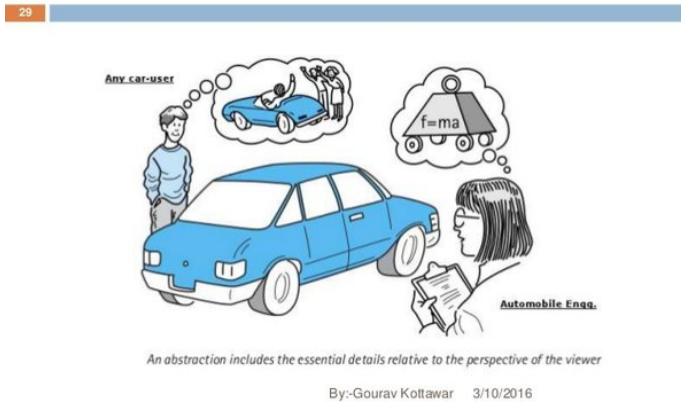
In Java, we use method overloading and method overriding to achieve polymorphism.

## ▼ Abstraction

*Hiding internal details and showing functionality* is known as abstraction. For example phone call, we don't know the internal processing.

In Java, we use abstract class and interface to achieve abstraction.

# Abstraction



## ▼ Encapsulation

**Binding (or wrapping) code and data together into a single unit are known as encapsulation.** For example, a capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.



## Capsule

## ▼ Features of Java language

### ▼ Object Oriented

In Java, everything is an Object. Java can be easily extended since it is based on the Object model.

### ▼ Platform independent

Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform independent byte

code. This byte code is distributed over the web and interpreted by virtual Machine (JVM) on whichever platform it is being run.

▼ Simple

Java is designed to be easy to learn. If you understand the basic concept of OOP, Java would be easy to master.

▼ Secure

With Java's secure feature, it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public-key encryption.

▼ Architectural-neutral

Java compiler generates an architecture-neutral object file format, which makes the compiled code to be executable on many processors, with the presence of Java runtime system.

▼ Portable

Being architectural-neutral and having no implementation dependent aspects of the specification makes Java portable. Compiler in Java is written in ANSI C with a clean portability boundary which is a POSIX subset.

▼ Robust

Java makes an effort to eliminate error prone situations by emphasizing mainly on compile time error checking and runtime checking.

▼ Multithread

With Java's multithreaded feature, it is possible to write programs that can do many tasks simultaneously. This design feature allows developers to construct smoothly running interactive applications.

▼ interpreted

Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and lightweight process.

▼ High performance

With the use of Just-In-Time compilers, Java enables high performance.

▼ Distributed

Java is designed for the distributed environment of the internet

#### ▼ Dynamic

Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry extensive amount of run-time information that can be

used to verify and resolve accesses to objects on run-time.

## ▼ Types of java programs

#### ▼ Java Platform, Standard Edition (Java SE)

Java SE platform consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits commonly used in Java technology applications.

#### ▼ Java Platform, Enterprise Edition (Java EE)

The Java EE platform is built on top of the Java SE platform. The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure network applications.

#### ▼ Java Platform, Micro Edition (Java ME)

The Java ME platform provides an API and a small-footprint virtual machine for running Java programming language applications on small devices, like **mobile phones**. The API is a subset of the Java SE API, along with special class libraries useful for **small device application development**. Java ME applications are often clients of Java EE platform services.

#### ▼ Java FX

Java FX technology is a platform for creating rich internet applications written in Java FX Script. Java FX Script is a statically-typed declarative language that is compiled to Java technology bytecode, which can then be run on a Java VM. Applications written for the Java FX platform can include and link to Java programming language classes, and may be clients of Java EE platform services.

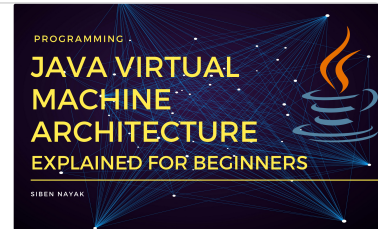
## ▼ java architecture

## 👉 JVM Architecture Full

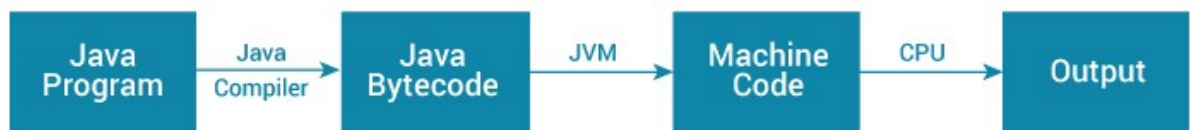
### JVM Tutorial - Java Virtual Machine Architecture Explained for Beginners

Whether you have used Java to develop programs or not, you might have heard about the Java Virtual Machine (JVM) at some point or another. JVM is the core of the Java ecosystem, and makes it possible for Java-based software programs to

 <https://www.freecodecamp.org/news/jvm-tutorial-java-virtual-machine-architecture-explained-for-beginners/>



Java Architecture combines the process of compilation and interpretation. It explains the various processes involved whilst formulating a Java program.



## ▼ Java Components

### ▼ Java Virtual Machine

JVM is responsible for running or executing the code. Takes '.class files' or the byte code from the java compiler and converts to machine code then interprets it give the output.

Java is platform-independent but **JVM is platform dependent**. since java follows **WORA (Write Once Run Anywhere)** principle java code is supported in any platforms.

### ▼ Java Runtime Environment



JRE (Java Runtime Environment) is a software package that provides Java class libraries, Java Virtual Machine (JVM), and other components that are required to run Java applications.

**JRE is the minimum requirement to run a java program.**

▼ **Java Development Kit**



JDK (Java Development Kit) is a software development kit required to develop applications in Java. *When you download JDK, JRE is also downloaded with it.*

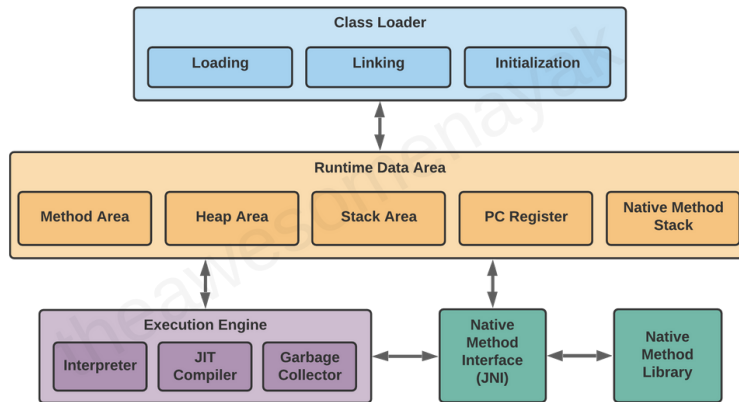
In addition to JRE, JDK also contains a number of development tools (compilers, JavaDoc, Java Debugger, etc. ).

▼ **What is java architecture?**

<https://youtu.be/QHIWkwxs0AI>

- In Java, there is a process of compilation and interpretation.
- The code written in Java, is converted into byte codes which is done by the Java Compiler.
- The byte codes, then are converted into machine code by the JVM.
- The Machine code is executed directly by the machine.





- The three main components of JVM architecture are :

#### ▼ **Class Loader**

Class loader is a subsystem of JVM which is used to load class files which are converted from .java files by the compiler. Whenever we run the java program, it is loaded first by the class loader.

The class Loader consists of Three main phases they are

#### ▼ **Loading**

The Three main class loaders are responsible for Loading :

- **Bootstrap Class Loader:** It is first class loader, which is responsible for loading class files from rt.jar also called runtime jar.

Eg : the 'System' in

```
System.out.print("Java");
```

is present in **rt.jar** file

- **Extension Class Loader:** It is responsible for loading the additional class files from (jre/lib/ext) folder.
- **Eg:** The third party connection files from databases like Oracle , Mongo DB etc. are found in the 'jre/lib/ext' folder the extension class loader is responsible for loading that files from folder to the memory
- **Application Class Loader:** Application class loader is the one that loads the class files from the application-specific jar. This is nothing but the application

that you created. Once your java program is compiled, the JVM creates class files. The Application class loader loads these class files to the memory area.

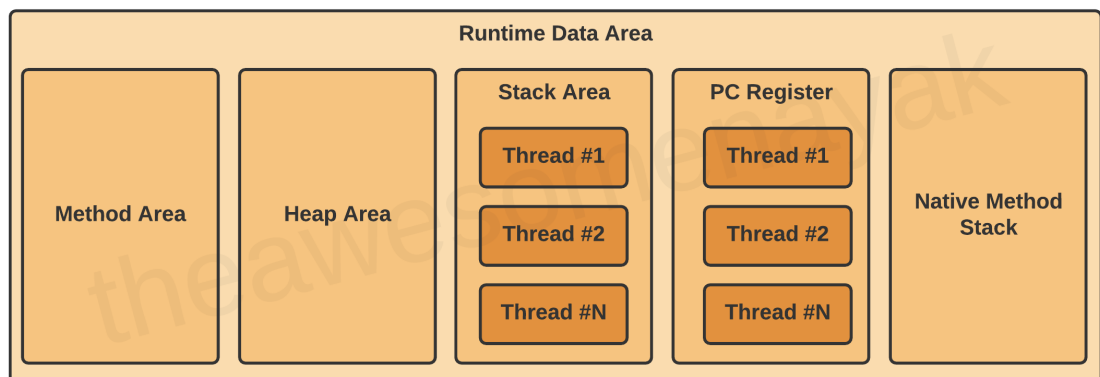
### ▼ Linking

- **Verify** : Once the class files are loaded to the memory, there is a verify phase where the bytecode class files are verified if they conform to standards.
- **Prepare**: In prepare phase, memory is allocated for the static variables and default values are assigned.
- **Resolve**: All the symbolic references are replaced with actual references

### ▼ Initialization

In the "Initialization phase" of class loader component, all the static variables are assigned with the actual values and it also executes the static initializers at this point of time

### ▼ Runtime Memory data area



- **Method/class Area**: The class level data are stored in this memory Area.

```
Class Employee{  
    static int count = 0;  
}
```

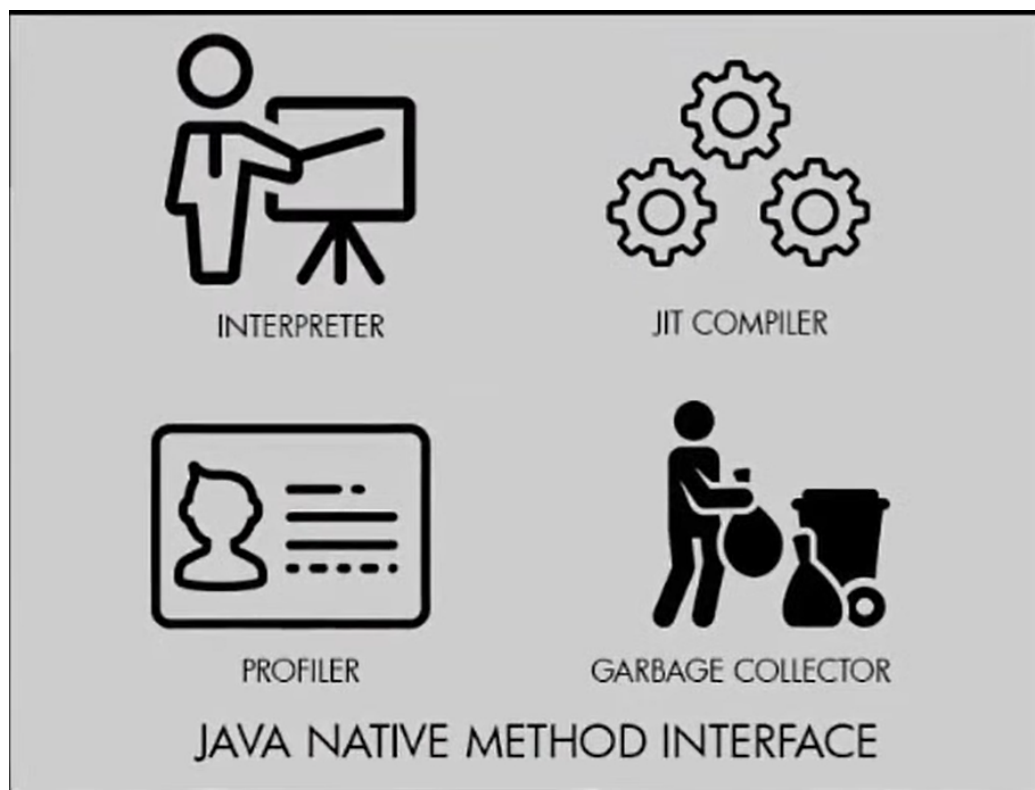
- **Heap memory** : All the objects and instance variables are stored in this memory.

```
new employee()
```

- **Stack Memory Area** : Local Variable , Operand Stack, Frame Data
- **PC Register** : This holds the current executing instructions.
- **Native Method Stack**: This holds the native method information.

#### ▼ Execution Engine

This is the actual engine that converts the bytecode to machine code and executes the instructions.



This part contains :

#### ▼ Interpreter

Interpreter is the one that reads the class files or bytecode and executes it one by one. The problem with the interpreter is that, when a method is called multiple times, it interprets those lines of bytecode again and again.

#### ▼ JIT Compiler

JIT compiler helps in overcoming the problem of the interpreter. When repeated method calls occur, JIT compiler compiles the bytecode to native code. This native

code will be used directly for repeated method calls. JIT compiler contains few components to achieve this feature

- **Intermediate code generator** : Generates Intermediate Code.
- **Code Optimizer** : Optimizes the intermediate code for better performance.
- **Target code generator** : converts intermediate code to native machine code .

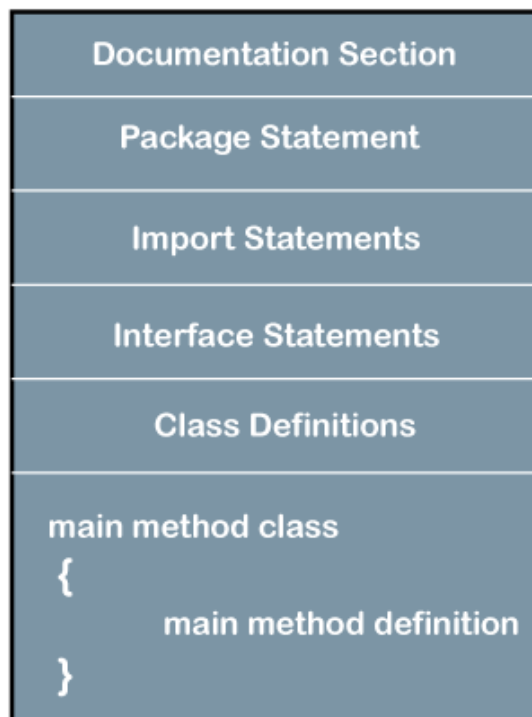
#### ▼ Profiler

Responsible for finding the hotspots, methods which are called repeatedly .

#### ▼ Garbage Collector

Garbage collector is responsible for destroying the objects that are no longer used.

## ▼ program structure



**Structure of Java Program**

A typical structure of a Java program contains the following elements:

## ▼ Documentation Section

The documentation section is an important section but optional for a Java program. It includes **basic information** about a Java program. The information can include the **author's name**, **date of creation**, **version**, **program name**, **company name**, and **description** of the program. It improves the readability of the program.

- **Single-line Comment:** It starts with a pair of forwarding slash (**//**)

```
//First Java Program
```

- **Multi-line Comment:** It starts with a **/\*** and ends with **\*/**. We write between these two symbols.

```
/*It is an example of  
multiline comment*/
```

- **Documentation Comment:** It starts with the delimiter (**/\*\***) and ends with **\*/**.

```
/**It is an example of documentation comment*/
```

## ▼ Package Declaration

The package declaration is optional. It is placed just after the documentation section. In this section, we declare the **package name** in which the class is placed. Note that there can be **only one package** statement in a Java program. It must be defined before any class and interface declaration. It is necessary because a Java class can be placed in different packages and directories based on the module they are used. For all these classes package belongs to a single parent directory. We use the keyword **package** to declare the package name.

```
package pkg; //where pkg is the package name  
or  
package com.pkg; //where com is the root directory and pkg is the subdir
```

## ▼ Import statements

The package contains the many predefined classes and interfaces. If we want to use any class of a particular package, we need to import that class. The import statement represents the class stored in the other package. We use the **import** keyword to import the class. It is written before the class declaration and after the package statement. We use the import statement in two ways, either import a specific class or import all classes of a particular package. In a Java program, we can use multiple import statements.

```
import java.util.Scanner; //it imports the Scanner class only
import java.util.*; //it imports all the class of the java.util package
```

## ▼ Interface statements

It is an optional section. We can create an **interface** in this section if required. We use the **interface** keyword to create an interface. An interface is a slightly different from the class. It contains only **constants** and **method** declarations. Another difference is that it cannot be instantiated. We can use interface in classes by using the **implements** keyword. An interface can also be used with other interfaces by using the **extends** keyword.

```
interface car
{
    void start();
    void stop();
}
```

## ▼ Class Definition

Class is a **vital** part of a Java program. Without the class, we cannot create any Java program. A Java program may contain more than one class definition. We use the **class** keyword to define the class. The class is a blueprint of a Java program. It contains information about user-defined methods, variables, and constants. Every Java program has at least one class that contains the main() method. For example:

```
class Student //class definition
{
}
```

## ▼ Class Variables and Constants

In this section, we define variables and **constants** that are to be used later in the program. In a Java program, the variables and constants are defined just after the class definition. The variables and constants store values of the parameters. It is used during the execution of the program. We can also decide and define the scope of variables by using the modifiers. It defines the life of the variables.

```
class Student //class definition
{
    String sname; //variable
    int id;
    double percentage;
}
```

## ▼ Main Methods Class

The **main() method**. It is essential for all Java programs. Because the execution of all Java programs starts from the main() method. In other words, it is an entry point of the class. It must be inside the class. Inside the main method, we create objects and call the methods. We use the following statement to define the main() method:

```
public static void main(String args[])
{
}
```

## ▼ Methods and behavior

We define the functionality of the program by using the methods. The methods are the set of instructions that we want to perform. These instructions execute at runtime and perform the specified task.

```
public class Demo //class definition
{
    public static void main(String args[])
    {
        void display()
        {
            System.out.println("Welcome to javatpoint");
        }
        //statements
    }
}
```

## ▼ Literals

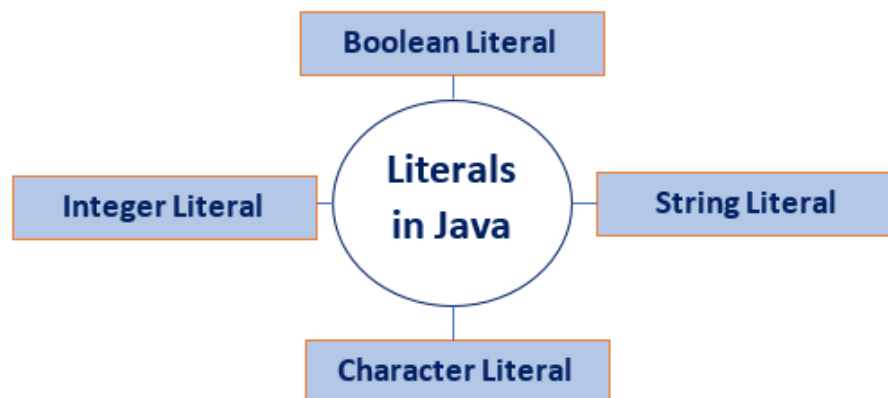
In Java, **literal** is a notation that represents a fixed value in the source code. In lexical analysis, literals of a given type are generally known as **tokens**. In this section, we will discuss the term **literals in Java**

`int cost = 340;`

↓                  ↓

**Variable    Literal**

Types of literals are :



### ▼ Integer Literal

Integer literals are sequences of digits. There are three types of integer literals:

- **Decimal Integer:** These are the set of numbers that consist of digits from 0 to 9. It may have a positive (+) or negative (-) Note that between numbers commas and non-digit characters are not permitted. For example, **5678**, **+657**, **-89**, etc.

```
int decVal = 26;
```



- **Octal Integer:** It is a combination of number have digits from 0 to 7 with a leading 0. For example, **045, 026,**

```
int octVal = 067;
```

- **Hexa-Decimal:** The sequence of digits preceded by **0x** or **0X** is considered as hexadecimal integers. It may also include a character from **a** to **f** or **A** to **F** that represents numbers from **10** to **15**, respectively. For example, **0xd, 0xf,**

```
int hexVal = 0x1a;
```

- **Binary Integer:** Base 2, whose digits consists of the numbers 0 and 1 (you can create binary literals in Java SE 7 and later). Prefix 0b represents the Binary system. For example, 0b11010.

```
int binVal = 0b11010;
```

## ▼ Character Literal

A character literal is expressed as a character or an escape sequence, enclosed in a **single** quote (") mark. It is always a type of char. For example, **'a', '%', '\u000d'**, etc.

**Single quote:** We can specify literal to a char data type as a single character within the single quote.

```
char ch = 'a';
```

**Char literal as Integral literal:** we can specify char literal as integral literal, which represents the Unicode value of the character, and that integral literal can be specified either in Decimal, Octal, and Hexadecimal forms. But the allowed range is 0 to 65535.

```
char ch = 062;
```

**Unicode Representation:** We can specify char literals in Unicode representation '\uxxxx'. Here xxxx represents 4 hexadecimal numbers.

```
char ch = '\u0061'; // Here /u0061 represent a.
```

**Escape Sequence:** Every escape character can be specified as char literals.

```
char ch = '\n';
```

```
// Java program to illustrate the
// application of char literals

public class Test {
    public static void main(String[] args)
    {
        // single character literal within single quote
        char ch = 'a';
        // It is an Integer literal with octal form
        char b = 0789;
        // Unicode representation
        char c = '\u0061';

        System.out.println(ch);
        System.out.println(b);
        System.out.println(c);

        // Escape character literal
        System.out.println "\"" is a symbol");
    }
}
```

### Output

```
a
error:Integer number too large
a
"  is a symbol
```

## ▼ Boolean Literal

Only two values are allowed for Boolean literals, i.e., true and false.

```
boolean b = true;
```

```
// Java program to illustrate the
// application of boolean literals

public class Test {
    public static void main(String[] args)
    {
        boolean b = true;
        boolean c = false;
        boolean d = 0;
        boolean b = 1;

        System.out.println(b);
        System.out.println(c);
        System.out.println(d);
        System.out.println(e);
    }
}
```

### Output

```
true
false
error: incompatible types: int cannot be converted to boolean
error: incompatible types: int cannot be converted to boolean
```

## ▼ String Literal

Any sequence of characters within double quotes is treated as String literals.

```
String s = "Hello";
```

```
// Java program to illustrate the
// application of String literals

public class Test {
    public static void main(String[] args)
```

```

{
    String s = "Hello";

    // If we assign without "" then it treats
    // as a variable and causes compiler error
    String s1 = Hello;

    System.out.println(s);
    System.out.println(s1);
}
}

```

### Output

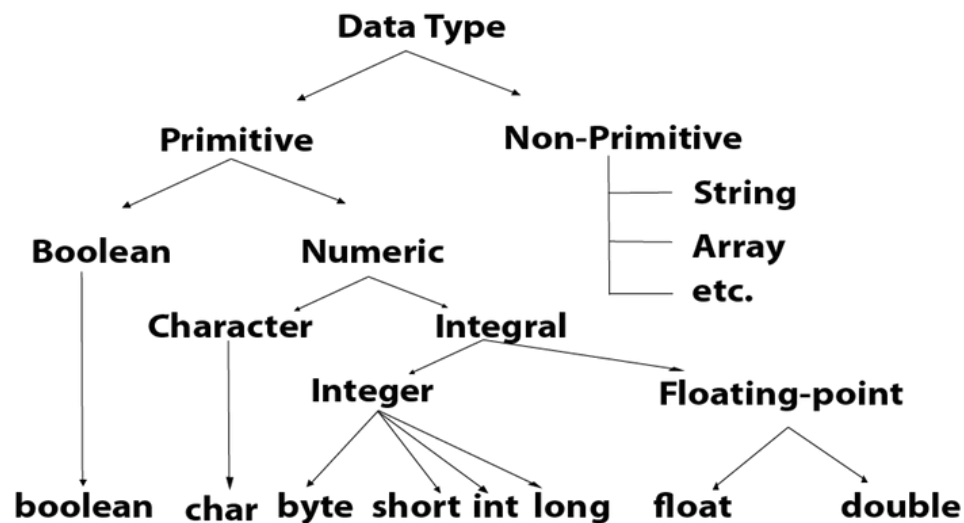
```

Hello
error: cannot find symbol
symbol:   variable Hello
location: class Test

```

## ▼ Data Types and variables

### ▼ Data Types



### ▼ Primitive Data type

#### 1. boolean type

- The `boolean` data type has two possible values, either `true` or `false`.

- Default value: `false`.
- They are usually used for **true/false** conditions.

```
class Main {
    public static void main(String[] args) {

        boolean flag = true;
        System.out.println(flag);    // prints true
    }
}
```

## 2. byte type

- The `byte` data type can have values from **128** to **127** (8-bit signed two's complement integer).
- If it's certain that the value of a variable will be within -128 to 127, then it is used instead of `int` to save memory.
- Default value: 0

```
class Main {
    public static void main(String[] args) {

        byte range;
        range = 124;
        System.out.println(range);    // prints 124
    }
}
```

## 3. short type

- The `short` data type in Java can have values from **32768** to **32767** (16-bit signed two's complement integer).
- If it's certain that the value of a variable will be within -32768 and 32767, then it is used instead of other integer data types ( `int` , `long` ).
- Default value: 0

```
class Main {
    public static void main(String[] args) {

        short temperature;
        temperature = -200;
        System.out.println(temperature);    // prints -200
    }
}
```

## 4. int type

- The `int` data type can have values from  $-2^{31}$  to  $2^{31}-1$  (32-bit signed two's complement integer).
- Default value 0

```
class Main {  
    public static void main(String[] args) {  
  
        int range = -4250000;  
        System.out.println(range); // print -4250000  
    }  
}
```

## 5. long type

- The `long` data type can have values from  $-2^{63}$  to  $2^{63}-1$  (64-bit signed two's complement integer).
- If you are using Java 8 or later, you can use an unsigned 64-bit integer with a minimum value of **0** and a maximum value of  $2^{64}-1$ .
- Default value: 0

## 6. double type

- The `double` data type is a double-precision 64-bit floating-point.
- It should never be used for precise values such as currency.
- Default value: 0.0 (0.0d).

```
class Main {  
    public static void main(String[] args) {  
  
        double number = -42.3;  
        System.out.println(number); // prints -42.3  
    }  
}
```

## 7. float type

- The `float` data type is a single-precision 32-bit floating-point.(numbers having decimal points).
- It should never be used for precise values such as currency.
- Default value: 0.0 (0.0f)

```
class Main {  
    public static void main(String[] args) {  
  
        float number = -42.3f;  
        System.out.println(number); // prints -42.3  
    }  
}
```

## 8. char type

- It's a 16-bit Unicode character.
- The minimum value of the char data type is `'\u0000'` (0) and the maximum value of the is `'\uffff'`.
- Default value: `'\u0000'`

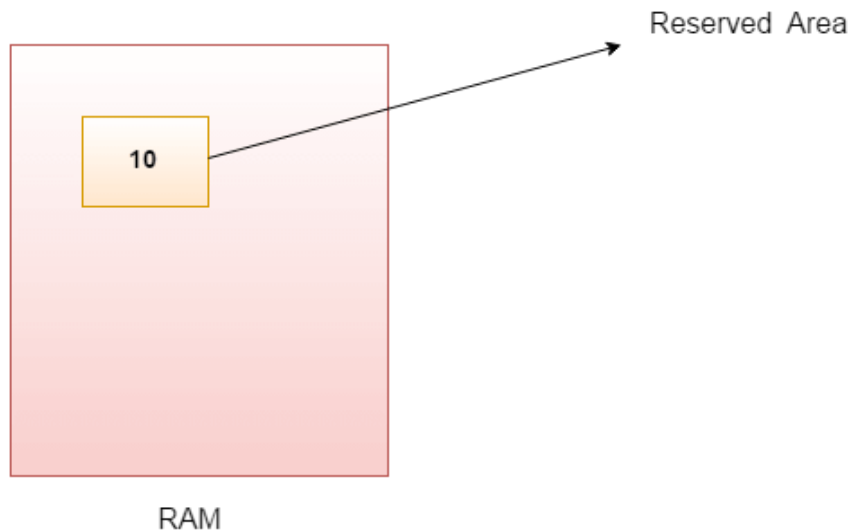
```
class Main {  
    public static void main(String[] args) {  
  
        char letter = '\u0051';  
        System.out.println(letter); // prints Q  
    }  
}
```

| Data Type            | Size    | Description                                                                       |
|----------------------|---------|-----------------------------------------------------------------------------------|
| <code>byte</code>    | 1 byte  | Stores whole numbers from -128 to 127                                             |
| <code>short</code>   | 2 bytes | Stores whole numbers from -32,768 to 32,767                                       |
| <code>int</code>     | 4 bytes | Stores whole numbers from -2,147,483,648 to 2,147,483,647                         |
| <code>long</code>    | 8 bytes | Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| <code>float</code>   | 4 bytes | Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits           |
| <code>double</code>  | 8 bytes | Stores fractional numbers. Sufficient for storing 15 decimal digits               |
| <code>boolean</code> | 1 bit   | Stores true or false values                                                       |

|      |         |                                                  |
|------|---------|--------------------------------------------------|
| char | 2 bytes | Stores a single character/letter or ASCII values |
|------|---------|--------------------------------------------------|

## ▼ Variables

A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.



```
int data=50;//Here data is variable
```

## ▼ Types of Variables

### ▼ Local Variable

A variable defined within a block or method or constructor is called a local variable.

- These variables are created when the block is entered, or the function is called and destroyed after exiting from the block or when the call returns from the function.
- The scope of these variables exists only within the block in which the variables are declared, i.e., we can access these variables only within that block.
- Initialization of the local variable is mandatory before using it in the defined scope.

### ▼ Instance Variable

Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.



- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used.
- Initialization of an instance variable is not mandatory. Its default value is 0.
- Instance variables can be accessed only by creating objects.

### ▼ Static Variable

Static variables are also known as class variables.

- These variables are declared similarly as instance variables. The difference is that static variables are declared using the static keyword within a class outside of any method, constructor or block.
- Unlike instance variables, we can only have one copy of a static variable per class, irrespective of how many objects we create.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Initialization of a static variable is not mandatory. Its default value is 0.
- If we access a static variable like an instance variable (through an object), the compiler will show a warning message, which won't halt the program. The compiler will replace the object name with the class name automatically.
- If we access a static variable without the class name, the compiler will automatically append the class name.

### example

```
public class A
{
    static int m=100;//static variable
    void method()
    {
        int n=90;//local variable
    }
    public static void main(String args[])
    {
        int data=50;//instance variable
    }
} //end of class
```

## ▼ Operators

**Operator** in Java is a symbol that is used to perform operations. For example: +, -, \*, / etc.

Types of java operators are :

### ▼ Unary Operators

The Java unary operators require only one operand. Unary operators are used to perform various operations i.e.:

- incrementing/decrementing a value by one
- negating an expression
- inverting the value of a boolean

▼ The unary operators are :

#### ▼ Unary Plus

It is used to represent positive values. Usually, we do not write the operator before the operand. Hence, it is optional.

```
+(operand)
```

#### ▼ Unary Minus

It is used to convert a positive value into a negative one.

```
-(operand)
```

#### ▼ Increment Operator

It is used to increment the value of an operand by one. The operator is represented by a pair of plus operators (++). The operator can be applied before or after an operand.

**Pre-increment Operator:** If an increment operator is written before (**prefix**) the operand is known as pre-increment. In such a case, the value is first incremented by 1 and then taken for the computing purpose.

```
++operand
```

**Example:**

Suppose x=9

y = ++x

then , y =10 and x =10

**Post-increment Operator:** If an increment operator is written after (**postfix**) the operand is known as post-increment. In such a case, the value is first processed and then incremented. It means the value that the variable holds first used for the computing purpose and then incremented by 1.

```
operand++
```

**Example:**

Suppose x=9

y = x++

then , y =9 and x =10

**▼ Decrement Operator**

It is used to decrement the value of an operand by 1. The operator is represented by a pair of minus operators (--). The operator can be applied before or after an operand.

**Pre-decrement Operator:** If a decrement operator is written before (**prefix**) the operand is known as pre-decrement. In such a case, the value is first decremented by 1 and then used for the computing purpose.

```
--operand
```

**Example:** Suppose x=9

y = --x then,

y =9 and x =9

**Post-decrement Operator:** If a decrement operator is written after (**postfix**) the operand is known as post-decrement. In such a case, the value is first used and then decrements by 1

```
operand--
```

**Example:** Suppose x=9

y = x- - then,

y =8 and x =9

### ▼ Logical Complement Operator

It is used to reverse the value of an operand. It means if an operand has value true, the complement of the operand will be false and vice-versa. It is represented by the exclamatory symbol (!).

!(operand)

| Name                        | Symbol | Description                                                | Example    | Equivalent Expression |
|-----------------------------|--------|------------------------------------------------------------|------------|-----------------------|
| Unary Plus                  | +      | It is used to represent the <b>positive</b> value.         | +a         | a                     |
| Unary Minus                 | -      | It is used to represent the <b>negative</b> value.         | -a         | -                     |
| Increment Operator          | ++     | It <b>increments</b> the value of a variable by <b>1</b> . | ++a or a++ | a=a+1                 |
| Decrement Operator          | --     | It <b>decrements</b> the value of a variable by <b>1</b> . | --a or a-- | a=a-1                 |
| Logical Complement Operator | !      | It <b>inverts</b> the value of a boolean variable.         | ! true     | -                     |

### ▼ Arithmetic Operator

Java arithmetic operators are used to perform addition, subtraction, multiplication, and division. They act as basic mathematical operations.

| Operators | Result                                                           |
|-----------|------------------------------------------------------------------|
| +         | Addition of two numbers                                          |
| -         | Subtraction of two numbers                                       |
| *         | Multiplication of two numbers                                    |
| /         | Division of two numbers                                          |
| %         | (Modulus Operator) Divides two numbers and returns the remainder |

**1. Addition(+):** This operator is a binary operator and is used to add two operands.

**Syntax:**

```
num1 + num2
```

**Example:**

```
num1 = 10, num2 = 20
sum = num1 + num2 = 30
```

**2. Subtraction(-):** This operator is a binary operator and is used to subtract two operands.

**Syntax:**

```
num1 - num2
```

**Example:**

```
num1 = 20, num2 = 10
sub = num1 - num2 = 10
```

**3. Multiplication(\*):** This operator is a binary operator and is used to multiply two operands.

**Syntax:**

```
num1 * num2
```

### Example:

```
num1 = 20, num2 = 10  
mult = num1 * num2 = 200
```

**4. Division(/):** This is a binary operator that is used to divide the first operand(dividend) by the second operand(divisor) and give the quotient as a result.

### Syntax:

```
num1 / num2
```

### Example:

```
num1 = 20, num2 = 10  
div = num1 / num2 = 2
```

**5. Modulus(%):** This is a binary operator that is used to return the remainder when the first operand(dividend) is divided by the second operand(divisor).

### Syntax:

```
num1 % num2
```

### Example:

```
num1 = 5, num2 = 2  
mod = num1 % num2 = 1
```

## ▼ Shift Operator

**shift operators** are the special type of operators that work on the bits of the data. These operators are used to shift the bits of the numbers from left to right or right to left depending on the type of shift operator used. There are three types of shift operators in Java:

### ▼ Signed Left Shift Operator (<<)

The signed left shift operator is a special type of operator used to move the bits of the expression to the left according to the number specified after the operator.

### Example

Consider  $x = 5$ .

Binary equivalent of 5 is 0101.

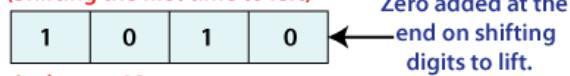


Assume that the statement is as follows

$x \ll 4$ :

Let  $y$  be 4.

At  $y=1$  (Shifting the first time to left)



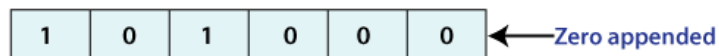
It is equivalent to 10.

At  $y = 2$



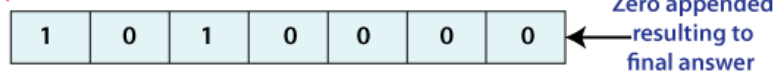
It is equivalent to 20.

At  $y = 3$



It is equivalent to 40.

At  $y = 4$



It is equivalent to 80.

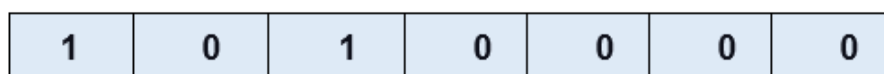
### ▼ Signed Right Shift Operator ( $\gg$ )

The signed right shift operator is a special type of operator used to move the bits of the expression to the right according to the number specified after the operator.

### Example

Consider  $x=80$ .

Binary equivalent of 80 is 1010000.



Assume that the statement is as follows:

$x \gg 4$ , let y be 4

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|

Assume that the statement is as follows

$x \gg 4$ :

Let y be 4.

At y=1 (Shifting the first time to the right)

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|

Zero discarded  
on shifting the  
bits to right

It is equivalent to 40.

At y = 2

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|

Again Zero  
discarded

It is equivalent to 20.

At y = 3

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 1 | 0 |
|---|---|---|---|

Zero discarded  
again

It is equivalent to 10.

At y = 4

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
|---|---|---|

Zero discarded  
resulting to  
the final answer

It is equivalent to 5.

### ▼ Unsigned Right Shift Operator ( $\gg$ )

The unsigned right shift operator is a special type of right shift operator that does not use the sign bit to fill in the sequence. The unsigned sign shift operator on the right always fills the sequence by 0.

### ▼ Relational Operator

**Java Relational Operators** are a bunch of binary operators used to check for relations between two operands, including equality, greater than, less than, etc. They return a boolean result after the comparison and are extensively used in looping statements as well as conditional if-else statements and so on. The general format of representing relational operator is:

**Syntax:**



```
variable1relation_operator variable2
```

### ▼ Operator 1: 'Equal to' operator (==)

This operator is used to check whether the two given operands are equal or not. The operator returns true if the operand at the left-hand side is equal to the right-hand side, else false.

#### Syntax:

```
var1 == var2
```

#### Illustration:

```
var1 = 20  
var2 = 20  
var1 == var2 results in true
```

### ▼ Operator 2: 'Not equal to' Operator(!=)

This operator is used to check whether the two given operands are equal or not. It functions opposite to that of the equal-to-operator. It returns true if the operand at the left-hand side is not equal to the right-hand side, else false.

#### Syntax:

```
var1 != var2
```

#### Illustration:

```
var1 = 12  
var2 = 20  
  
var1 != var2 results in true
```

### ▼ Operator 3: 'Greater than' operator(>)

This checks whether the first operand is greater than the second operand or not. The operator returns true when the operand at the left-hand side is greater than the right-hand side.

#### Syntax:

```
var1 > var2
```

**Illustration:**

```
var1 = 30  
var2 = 20  
  
var1 > var2 results in true
```

▼ **Operator 4: 'Less than' Operator(<)**

This checks whether the first operand is less than the second operand or not. The operator returns true when the operand at the left-hand side is less than the right-hand side. It functions opposite to that of the greater-than operator.

**Syntax:**

```
var1 < var2
```

**Illustration:**

```
var1 = 10  
var2 = 20  
  
var1 < var2 results in true
```

▼ **Operator 5: Greater than or equal to (>=)**

This checks whether the first operand is greater than or equal to the second operand or not. The operator returns true when the operand at the left-hand side is greater than or equal to the right-hand side.

**Syntax:**

```
var1 >= var2
```

**Illustration:**

```
var1 = 20  
var2 = 20  
var3 = 10
```

```
var1 >= var2 results in true
var2 >= var3 results in true
```

## ▼ Operator 6: Less than or equal to (<=)

This checks whether the first operand is less than or equal to the second operand or not. The operator returns true when the operand at the left-hand side is less than or equal to the right-hand side.

### Syntax:

```
var1 <= var2
```

### Illustration:

```
var1 = 10
var2 = 10
var3 = 9

var1 <= var2 results in true
var2 <= var3 results in false
```

## Java program to illustrate relational operator

```
public class Test {

    public static void main(String args[]) {
        int a = 10;
        int b = 20;

        System.out.println("a == b = " + (a == b) );
        System.out.println("a != b = " + (a != b) );
        System.out.println("a > b = " + (a > b) );
        System.out.println("a < b = " + (a < b) );
        System.out.println("b >= a = " + (b >= a) );
        System.out.println("b <= a = " + (b <= a) );
    }
}
```

## Output

```
a == b = false
a != b = true
a > b = false
a < b = true
```

```
b >= a = true
b <= a = false
```

## ▼ Bitwise Operator

Bitwise operators are used to performing the manipulation of individual bits of a number. They can be used with any integral type (char, short, int, etc.). They are used when performing update and query operations of the Binary indexed trees.

### 1. Bitwise OR (|)

This operator is a binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e., if either of the bits is 1, it gives 1, else it shows 0.

**Example:**

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise OR Operation of 5 and 7
  0101
| 0111
-----
  0111 = 7 (In decimal)
```

### 2. Bitwise AND (&)

This operator is a binary operator, denoted by '&.' It returns bit by bit AND of input values, i.e., if both bits are 1, it gives 1, else it shows 0.

**Example:**

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise AND Operation of 5 and 7
  0101
& 0111
-----
  0101 = 5 (In decimal)
```

### 3. Bitwise XOR (^)

This operator is a binary operator, denoted by '^.' It returns bit by bit XOR of input values, i.e., if corresponding bits are different, it gives 1, else it shows 0.

**Example:**

```

a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise XOR Operation of 5 and 7
  0101
^ 0111
-----
  0010 = 2 (In decimal)

```

#### 4. Bitwise Complement (~)

This operator is a unary operator, denoted by '~'. It returns the one's complement representation of the input value, i.e., with all bits inverted, which means it makes every 0 to 1, and every 1 to 0.

##### Example:

```

a = 5 = 0101 (In Binary)

Bitwise Complement Operation of 5
~ 0101
-----
 1010 = 10 (In decimal)

```

#### java program to demonstrate Bitwise operator example

```

// Java program to illustrate
// bitwise operators

public class operators {
    public static void main(String[] args)
    {
        // Initial values
        int a = 5;
        int b = 7;

        // bitwise and
        // 0101 & 0111=0101 = 5
        System.out.println("a&b = " + (a & b));

        // bitwise or
        // 0101 | 0111=0111 = 7
        System.out.println("a|b = " + (a | b));

        // bitwise xor
        // 0101 ^ 0111=0010 = 2
        System.out.println("a^b = " + (a ^ b));

        // bitwise not
        // ~00000000 00000000 00000000 00000101=11111111 11111111 11111111 11111010
    }
}

```

```

        // will give 2's complement (32 bit) of 5 = -6
        System.out.println("~a = " + ~a);

        // can also be combined with
        // assignment operator to provide shorthand
        // assignment
        // a=a&b
        a &= b;
        System.out.println("a= " + a);
    }
}

```

## Output

```

a&b = 5
a|b = 7
a^b = 2
~a = -6
a= 5

```

## ▼ Logical Operator

The operators that are used to do logical operators ,

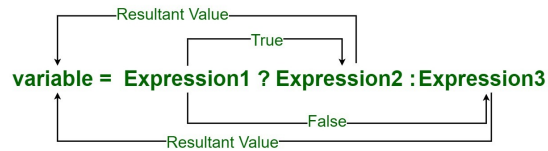
The types of Logical operators are :

| Operator | Name        | Description                                             | Example                    |
|----------|-------------|---------------------------------------------------------|----------------------------|
| &&       | Logical and | Returns true if both statements are true                | $x < 5 \ \&\& \ x < 10$    |
|          | Logical or  | Returns true if one of the statements is true           | $x < 5 \    \ x < 4$       |
| !        | Logical not | Reverse the result, returns false if the result is true | $!(x < 5 \ \&\& \ x < 10)$ |

## ▼ Ternary Operator

Java ternary operator is the only conditional operator that takes three operands. It's a one-liner replacement for the if-then-else statement and is used a lot in Java programming. We can use the ternary operator in place of if-else conditions or even switch conditions using nested ternary operators.

### Conditional or Ternary Operator (?:) in Java



#### Syntax:

```
variable = Expression1 ? Expression2 : Expression3
```

If operates similarly to that of the if-else statement as in *Expression2* is executed if *Expression1* is true else *Expression3* is executed.

```
if(Expression1)
{
    variable = Expression2;
}
else
{
    variable = Expression3;
}
```

#### ▼ Assignment Operator.

Assignment operators are used in Java to assign values to variables

| Operator | Example | Equivalent to |
|----------|---------|---------------|
| =        | a = b;  | a = b;        |
| +=       | a += b; | a = a + b;    |
| -=       | a -= b; | a = a - b;    |
| *=       | a *= b; | a = a * b;    |
| /=       | a /= b; | a = a / b;    |
| %=       | a %= b; | a = a % b;    |

## ▼ Control statements

Java uses control statements to control the flow of execution of a program based on certain conditions. These are used to cause the flow of execution to advance and branch based on changes to the state of a program.

There are 3 Control statements in Java they are:

## ▼ **Decision-Making statements**

Decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java,.

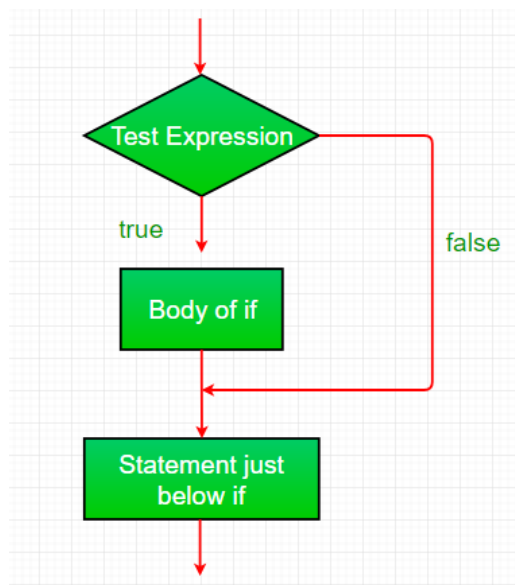
### ▼ **if statements**

The "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false

Types of if Statements are

#### ▼ **Simple if statement**

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.



### **Syntax**



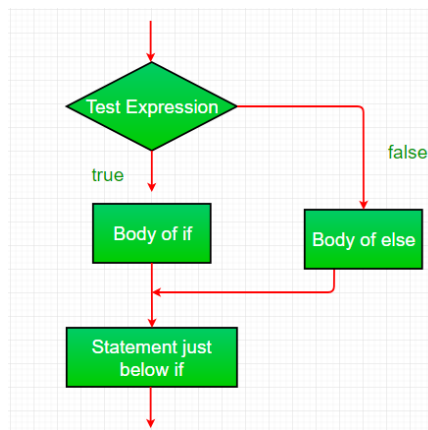
```
if(condition) {  
    statement 1; //executes when condition is true  
}
```

### example

```
// Java program to illustrate If statement  
import java.util.*;  
  
class IfDemo {  
    public static void main(String args[])  
    {  
        int i = 10;  
  
        if (i < 15)  
            System.out.println("Inside If block");  
            System.out.println("10 is less than 15");  
        // This statement will be executed  
        // as if considers one statement by default  
        System.out.println("I am Not in if");  
    }  
}
```

### ▼ if-else statement

he if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else if the condition is false. Here comes the else statement. We can use the else statement with if statement to execute a block of code when the condition is false.



### **Syntax:**

```
if (condition)  
{
```

```
        // Executes this block if
        // condition is true
    }
    else
    {
        // Executes this block if
        // condition is false
    }
}
```

### **Example**

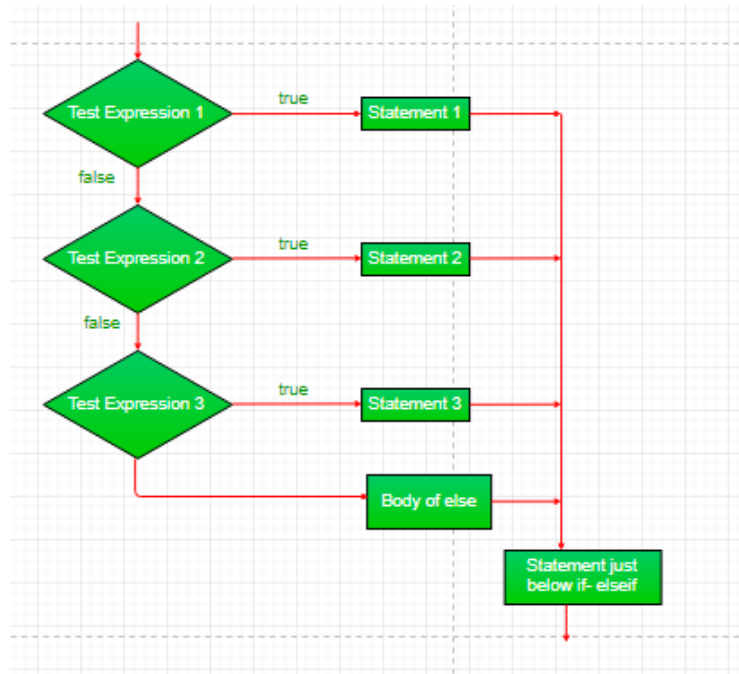
```
// Java program to illustrate if-else statement
import java.util.*;

class IfElseDemo {
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
            System.out.println("i is smaller than 15");
        else
            System.out.println("i is greater than 15");
    }
}
```

#### ▼ **if-else-if ladder**

Here, a user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.



## Syntax

```
if (condition)
    statement;
else if (condition)
    statement;
.
.
else
    statement;
```

## Example

```
// Java program to illustrate if-else-if ladder
import java.util.*;

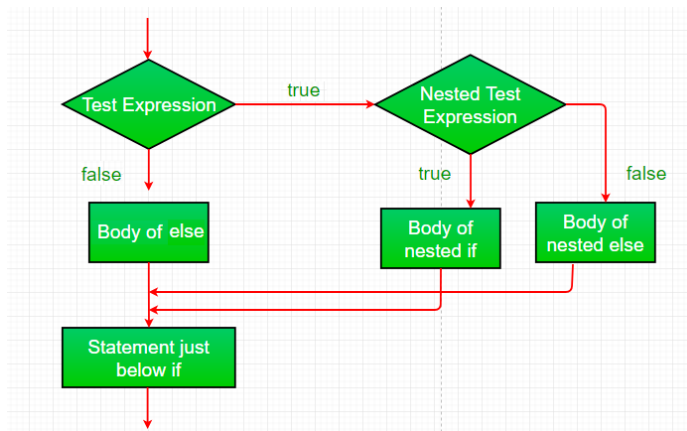
class ifelseifDemo {
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)
            System.out.println("i is 20");
        else
            System.out.println("i is not present");
    }
}
```

```
}  
}
```

### ▼ Nested if-statement

A nested if is an if statement that is the target of another if or else. Nested if statements mean an if statement inside an if statement. Yes, java allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.



### Syntax:

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
}
```

### Example

```
// Java program to illustrate nested-if statement
import java.util.*;

class NestedIfDemo {
    public static void main(String args[])
    {
        int i = 10;

        if (i == 10 || i<15) {
            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");
        }
    }
}
```

```

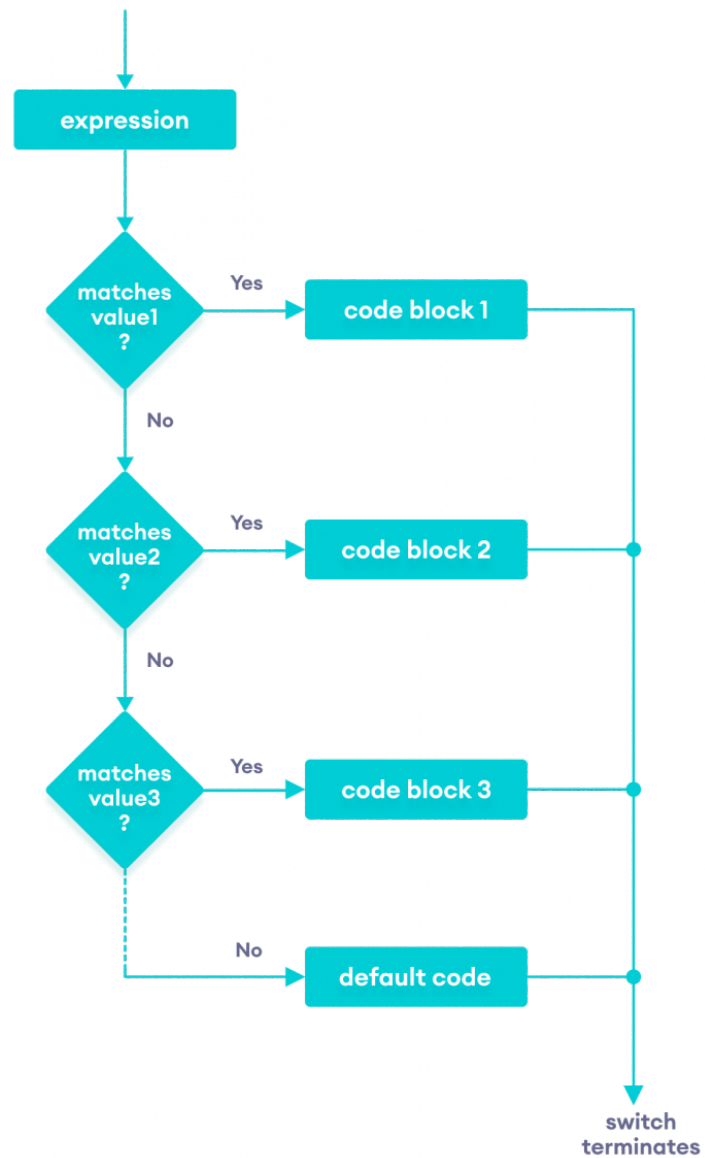
// Nested - if statement
// Will only be executed if statement above
// it is true
if (i < 12)
    System.out.println(
        "i is smaller than 12 too");
} else{
    System.out.println("i is greater than 15");
}
}
}

```

## ▼ Switch

The switch statement is a multi-way branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.

- The case variables can be int, short, byte, char, or enumeration. String type is also supported since version 7 of Java
- Cases cannot be duplicate
- Default statement is executed when any of the case doesn't match the value of expression. It is optional.
- Break statement terminates the switch block when the condition is satisfied. It is optional, if not used, next case is executed.
- While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value.



### Syntax:

```
switch (expression)
{
    case value1:
        statement1;
        break;
    case value2:
        statement2;
        break;
    .
    .
    case valueN:
        statementN;
```

```

        break;
    default:
        statementDefault;
}

```

### Example

```

// Java Program to check the size
// using the switch...case statement

class Main {
    public static void main(String[] args) {

        int number = 44;
        String size;

        // switch statement to check size
        switch (number) {

            case 29:
                size = "Small";
                break;

            case 42:
                size = "Medium";
                break;

            // match the value of week
            case 44:
                size = "Large";
                break;

            case 48:
                size = "Extra Large";
                break;

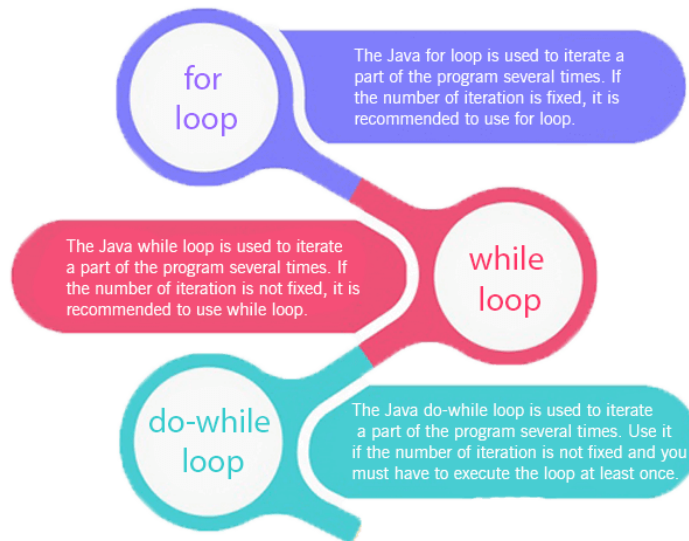
            default:
                size = "Unknown";
                break;

        }
        System.out.println("Size: " + size);
    }
}

```

## ▼ Loop Statements

Looping is a feature which facilitates the execution of a set of instructions/functions repeatedly while some condition evaluates to true. Java provides three ways for executing the loops.



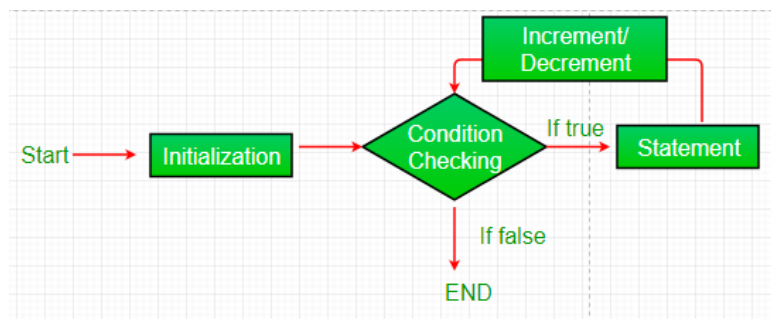
The three types of java loops are :

### ▼ For Loop(Entry Controlled Loop)

For loop provides a concise way of writing the loop structure. Unlike a while loop, a for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping.

**Syntax:**

```
for (initialization condition; testing condition; incrmnt/decrmnt)
{
    statement(s)
}
```



- **Initialization condition:** Here, we initialize the variable in use. It marks the start of a for loop. An already declared variable can be used or a variable can be declared, local to loop only.



- **Testing Condition:** It is used for testing the exit condition for a loop. It must return a boolean value. It is also an **Entry Control Loop** as the condition is checked prior to the execution of the loop statements.
- **Statement execution:** Once the condition is evaluated to true, the statements in the loop body are executed.
- **Increment/ Decrement:** It is used for updating the variable for next iteration.
- **Loop termination:** When the condition becomes false, the loop terminates marking the end of its life cycle.

### Example

```
/*package whatever //do not write package name here */

import java.io.*;

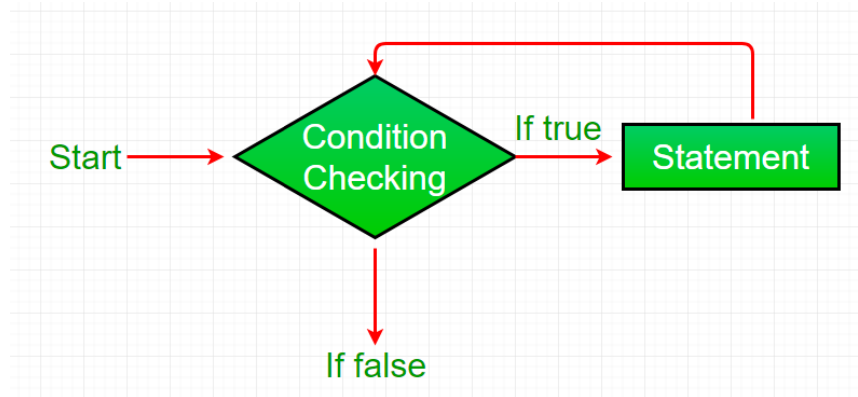
class GFG {
    public static void main (String[] args) {
        for (int i=0;i<=10;i++)
        {
            System.out.println(i);
        }
    }
}
```

### ▼ While Loop(Entry controlled Loop)

A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

#### **Syntax :**

```
while (boolean condition)
{
    loop statements...
}
```



- While loop starts with the checking of Boolean condition. If it evaluated to true, then the loop body statements are executed otherwise first statement following the loop is executed. For this reason it is also called **Entry control loop**
- Once the condition is evaluated to true, the statements in the loop body are executed. Normally the statements contain an update value for the variable being processed for the next iteration.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.

### Example

```
/*package whatever //do not write package name here */  
  
import java.io.*;  
  
class GFG {  
    public static void main (String[] args) {  
        int i=0;  
        while (i<=10)  
        {  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```

### **Output**

```
0  
1  
2  
3  
4  
5  
6
```

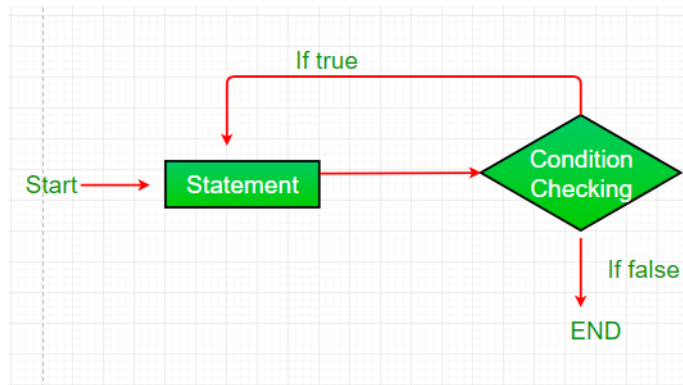
```
7  
8  
9  
10
```

## ▼ Do-While(Exit Controlled Loop)

**do while** loop is similar to while loop with only difference that it checks for condition after executing the statements, and therefore is an example of **Exit Control Loop**.

### Syntax:

```
do  
{  
    statements..  
}  
while (condition);
```



- do while loop starts with the execution of the statement(s). There is no checking of any condition for the first time.
- After the execution of the statements, and update of the variable value, the condition is checked for true or false value. If it is evaluated to true, next iteration of loop starts. When the condition becomes false, the loop terminates which marks the end of its life cycle.
- It is important to note that the do-while loop will execute its statements atleast once before any condition is checked, and therefore is an example of **exit control loop**.

### Example

```
/*package whatever //do not write package name here */
```

```
import java.io.*;

class GFG {
    public static void main (String[] args) {
        int i=0;
        do
        {
            System.out.println(i);
            i++;
        }while(i<=10);
    }
}
```

## Output

```
0
1
2
3
4
5
6
7
8
9
10
```

## ▼ Jump Statements

Jumping statements are control statements that transfer execution control from one point to another point in the program. There are two Jump statements that are provided in the Java programming language:

### ▼ Break Statement

#### 1. Using Break Statement to exit a loop:

In java, the break statement is used to terminate the execution of the nearest looping statement or switch statement. The break statement is widely used with the switch statement, **for** loop, **while** loop, **do-while** loop.

#### Syntax

```
break;
```

When a **break** statement is found inside a loop, the loop is terminated, and the control reaches the statement that follows the loop.

```
// Java program to illustrate the
// break keyword in Java
import java.io.*;

class GFG {
    public static void main(String[] args)
    {
        int n = 10;
        for (int i = 0; i < n; i++) {
            if (i == 6)
                break;
            System.out.println(i);
        }
    }
}
```

## Output

```
0
1
2
3
4
5
```

As you see, the code is meant to print 1 to 10 numbers using for loop, but it prints only 1 to 5 . as soon as i is equal to 6, the control terminates the loop.

In a switch statement, if the **break** statement is missing, every **case** label is executed till the end of the switch.

## 2. Use Break as a form of goto

Java does not have a **goto** statement because it produces an unstructured way to alter the flow of program execution. Java illustrates an extended form of the break statement. This form of **break** works with **the label**. The label is the name of a label that identifies a statement or a block of code.

### Syntax:

```
break label;
```

When this form of **break** executes, control jumps out of the labeled statement or block.

### Example

```
// Java program to illustrate the
// break keyword as a Goto statement in Java
import java.io.*;

class GFG {
    public static void main(String[] args)
    {
        for (int i = 0; i < 3; i++) {
            one : { // label one
                two : { // label two
                    three : { // label three
                        System.out.println("i=" + i);
                        if (i == 0)
                            break one; // break to label one
                        if (i == 1)
                            break two; // break to label two
                        if (i == 2)
                            break three; // break to label three
                    }
                    System.out.println("after label three");
                }
                System.out.println("after label two");
            }
            System.out.println("after label one");
        }
    }
}
```

## Output

```
i=0
after label one
i=1
after label two
after label one
i=2
after label three
after label two
after label one
```

In the above program, when  $i=0$ , the first if statement succeeds, and cause a break to label one and then prints the statement. When  $i=1$ , the second if statement succeeds, and cause a break to label two and then prints the statements. When  $i=2$ , the third if statement succeeds, and cause a break to the to label three and then prints all the three statements.

## ▼ Continue Statement

The **continue** statement pushes the next repetition of the loop to take place, hopping any code between itself and the conditional expression that controls the loop.

## Example

```
// Java program to illustrate the
// continue keyword in Java
import java.io.*;

class GFG {
    public static void main(String[] args)
    {
        for (int i = 0; i < 10; i++) {
            if (i == 6){
                System.out.println();
                // using continue keyword
                // to skip the current iteration
                continue;
            }
            System.out.println(i);
        }
    }
}
```

## Output

```
0
1
2
3
4
5

7
8
9
```

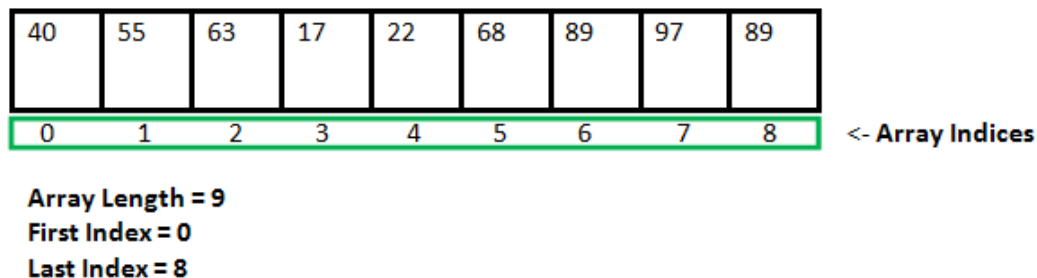
In the program, when the value of *i* is 6, the compiler encounters the `continue` statement, then 6 is skipped.

## ▼ Arrays

**Array in java** is a group of like-typed variables referred to by a common name.

- In Java, all arrays are dynamically allocated.
- Arrays are stored in contiguous memory [consecutive memory locations].
- Since arrays are objects in Java, we can find their length using the object property *length*. This is different from C/C++, where we find length using `sizeof`.
- A Java array variable can also be declared like other variables with `[]` after the data type.

- The variables in the array are ordered, and each has an index beginning from 0.
- Java array can also be used as a static field, a local variable, or a method parameter.
- The **size** of an array must be specified by int or short value and not long.
- The direct superclass of an array type is **Object**.
- Every array type implements the interfaces Cloneable and **java.io.Serializable**.
- This storage of arrays helps us randomly accessing the elements of an array [Support Random Access].
- The size of the array cannot be altered(once initialized). However, an array reference can be made to point to another array.
- An array can contain primitives (int, char, etc.) and object (or non-primitive) references of a class depending on the definition of the array. In the case of primitive data types, the actual values are stored in contiguous memory locations. In the case of class objects, **the actual objects are stored in a heap segment**



## ▼ Creating an Array (one Dimensional Array)

The general form of a one-dimensional array declaration is

```
type var-name[];  
OR  
type[] var-name;
```

*An array declaration has two components: the type and the name. type declares the element type of the array. The element type determines the data type of each element that comprises the array. Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc., or user-defined data types (objects of a class). Thus, the element type for the array determines what type of data the array will hold.*



## Example

```
int Array1[];  
or  
int[] Array1;  
//Array 1 is the array name
```

### ▼ Initializing an Array(One Dimensional Array)

When an array is declared, only a reference of an array is created. To create or give memory to the array, you create an array using `new` keyword.

#### Syntax

```
var-name = new type [size];
```

Here, *type* specifies the type of data being allocated, *size* determines the number of elements in the array, and *var-name* is the name of the array variable that is linked to the array. To use *new* to allocate an array, **we must specify the type and number of elements to allocate.**

#### **Example:**

```
int intArray[];    //declaring array  
intArray = new int[20]; // allocating memory to array
```

OR

```
int[] intArray = new int[20]; // combining both statements in one
```

---

### **Array Literal**

In a situation where the size of the array and variables of the array are already known, array literals can be used.

```
int[] intArray = new int[]{ 1,2,3,4,5,6,7,8,9,10 };  
// Declaring array literal
```

- The length of this array determines the length of the created array.
- There is no need to write the new int[] part in the latest versions of Java.

## ▼ Accessing Java Array Elements using for Loop

Each element in the array is accessed via its index. The index begins with 0 and ends at (total array size)-1. All the elements of array can be accessed using Java for Loop.

### Syntax

```
// accessing the elements of the specified array
for (int i = 0; i < arr.length; i++)
    System.out.println("Element at index " + i + " : " + arr[i]);
```

### Example

```
// Java program to illustrate creating an array
// of integers, puts some values in the array,
// and prints each value to standard output.

class GFG {
    public static void main(String[] args)
    {
        // declares an Array of integers.
        int[] arr;

        // allocating memory for 5 integers.
        arr = new int[5];

        // initialize the first elements of the array
        arr[0] = 10;

        // initialize the second elements of the array
        arr[1] = 20;

        // so on...
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        // accessing the elements of the specified array
        for (int i = 0; i < arr.length; i++)
            System.out.println("Element at index " + i
                               + " : " + arr[i]);
    }
}
```

### **Output**

```
Element at index 0 : 10
Element at index 1 : 20
Element at index 2 : 30
Element at index 3 : 40
Element at index 4 : 50
```

## ▼ Classes and objects

### ▼ Class

1. Class is a set of object which shares common characteristics/ behavior and common properties/ attributes.
2. Class is not a real world entity. It is just a template or blueprint or prototype from which objects are created.
3. Class does not occupy memory.
4. Class is a group of variables of different data types and group of methods.

A class in java can contain:

- data member
- method
- constructor
- nested class
- interface

### ▼ Defining a class

Java provides a reserved keyword **class** to define a class. The keyword must be followed by the class name. Inside the class, we declare methods and variables.

In general, class declaration includes the following in the order as it appears:

1. **Modifiers:** A class can be public or has default access.
2. **class keyword:** The class keyword is used to create a class.
3. **Class name:** The name must begin with an initial letter (capitalized by convention).
4. **Super-class (if any):** The name of the class's parent (superclass), if any, preceded by the keyword `extends`. A class can only extend (subclass) one parent.

5. **Interfaces (if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.
6. **Body:** The class body surrounded by braces, { }.

**Syntax:**

```
<access specifier> class class_name
{
    // member variables
    // class methods
}
```

**Example**

```
public class Dog {
    String breed;
    int age;//these are instance variables
    String color;
    static String food;//Static variable

    void barking() {
        int local_var;//Local variable
    }

    void hungry() {
    }

    void sleeping() {
    }
}
```

A class can contain any of the following variable types.

- **Local variables** – Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables** – Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- **Class variables or Static Variables**– Class variables are variables declared within a class, outside any method, with the static keyword.

## ▼ Objects

An Object is a real world entity , also the it is a blueprint of a class , an object must contain :

1. **State:** It is represented by attributes of an object. It also reflects the properties of an object.
2. **Behavior:** It is represented by methods of an object. It also reflects the response of an object with other objects.
3. **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

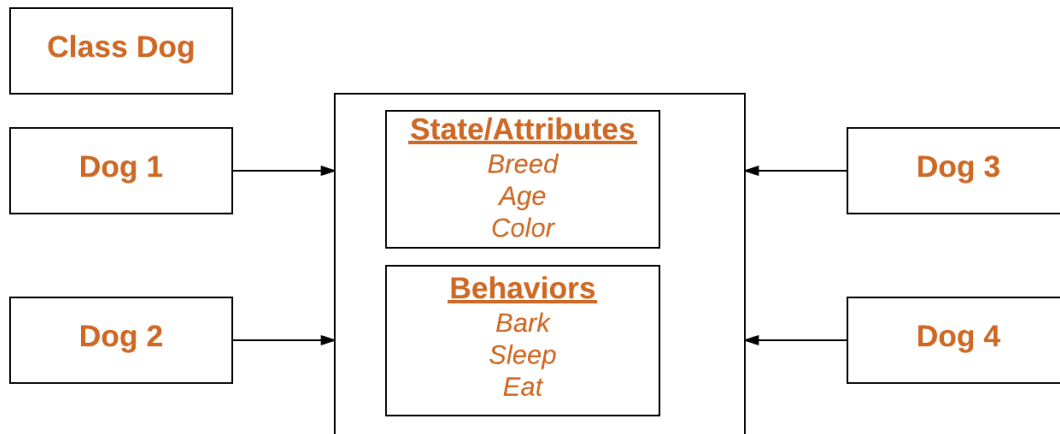
There are three steps when creating an object from a class –

- **Declaration** – A variable declaration with a variable name with an object type.
- **Instantiation** – The 'new' keyword is used to create the object.
- **Initialization** – The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

### Syntax

```
ClassName object = new ClassName();
```

When an object of a class is created, the class is said to be **instantiated**. All the instances share the attributes and the behavior of the class. But the values of those attributes, i.e. the state are unique for each object. A single class may have any number of instances.



As we declare variables like (type name;). This notifies the compiler that we will use the name to refer to data whose type is type. With a primitive variable, this declaration also reserves the proper amount of memory for the variable. So for reference variable, the type must be strictly a concrete class name. In general, we **can't** create objects of an abstract class or an interface.

### Initializing an object

The new operator instantiates a class by allocating memory for a new object and returning a reference to that memory. The new operator also invokes the class constructor.

```
public class Dog {

    int age;

    //This is a constructor
    public Dog(int k) {
        age = k;
    }

    void DogAge() {
        System.out.println("Age Of Dog = " + age);
    }

    public static void main(String[] args) {

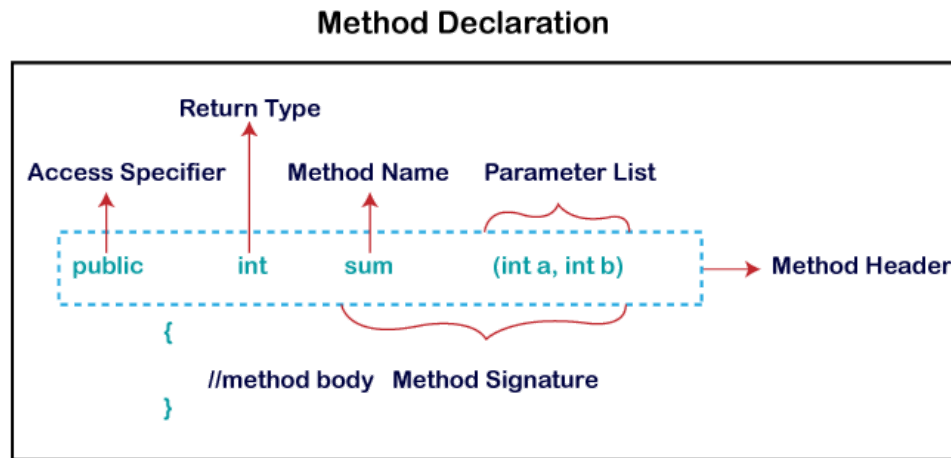
        // Instatiating An object
        Dog d = new Dog(5);

        d.DogAge();
    }

}
```

## ▼ Method declaration

The method declaration provides information about method attributes, such as visibility, return-type, name, and arguments. It has six components that are known as **method header**.



**Method Signature:** Every method has a method signature. It is a part of the method declaration. It includes the **method name** and **parameter list**.

**Access Specifier:** Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides **four** types of access specifier:

- **Public:** The method is accessible by all classes when we use public specifier in our application.
- **Private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **Protected:** When we use protected access specifier, the method is accessible within the same package or sub-classes in a different package.
- **Default:** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

**Return Type:** Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use void keyword.

**Method Name:** It is a unique name that is used to define the name of a method. It must be corresponding to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be **subtraction()**. A method is invoked by its name.

**Parameter List:** It is the list of parameters separated by a comma and enclosed in the pair of parentheses. It contains the data type and variable name. If the method has no parameter, left the parentheses blank.

**Method Body:** It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within the pair of curly braces.

## ▼ Constructor

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

Every time an object is created using the `new` keyword, at least one constructor is called.

It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

```
public class Dog {  
  
    int age;  
  
    //This is a constructor  
    public Dog(int k) {  
        age = k;  
    }  
  
    void DogAge() {  
        System.out.println("Age Of Dog = " + age);  
    }  
  
    public static void main(String[] args) {  
  
        // Instatiating An object  
        Dog d = new Dog(5);  
  
        d.DogAge();  
    }  
}
```

.

### Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized



## Types of java Constructors:

### ▼ Default Constructor

A constructor that has no parameters is known as default the constructor. A default constructor is invisible. And if we write a constructor with arguments or no arguments then the compiler does not create a default constructor. It is taken out. It is being overloaded and called a parameterized constructor. The default constructor changed into the parameterized constructor. But Parameterized constructor can't change the default constructor.

#### Syntax

```
<class_name>(){ } //no need to write this code in program to invoke default constructor
```

#### Example

```
import java.io.*;

class GFG {
    GFG() { System.out.println("Default constructor"); }
    public static void main(String[] args)
    {
        GFG hello = new GFG();
    }
}
```

#### **Output**

```
Default constructor
```

**Note:** Default constructor provides the default values to the object like 0, null, etc. depending on the type.

### ▼ Non-Parameterized Constructor

A constructor that has no parameter is known as the Non-parameterized Constructor or Zero argument constructor. If we don't define a constructor in a class, then the compiler creates a **constructor(with no arguments)**

for the class. And if we write a constructor with arguments or no arguments then the compiler does not create a default constructor.

```
// Java Program to illustrate calling a
// Zero argument constructor

import java.io.*;

class Computer {
    int num;
    String name;

    // this would be invoked while an object
    // of that class is created.
    Computer() { System.out.println("Constructor called"); }
}

class main_class {
    public static void main(String[] args)
    {
        // this would invoke default constructor.
        Computer g = new Computer();

        // Default constructor provides the default
        // values to the object like 0, null
        System.out.println(g.name);
        System.out.println(g.num);
    }
}
```

## Output

```
Constructor called
null
0
```

## ▼ Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

```
//Java Program to demonstrate the use of the parameterized constructor.
class Student4{
    int id;
    String name;
    //creating a parameterized constructor
    Student4(int i,String n){
        id = i;
        name = n;
    }
    //method to display the values
```

```

    void display(){System.out.println(id+" "+name);}

    public static void main(String args[]){
        //creating objects and passing values
        Student4 s1 = new Student4(111,"Karan");
        Student4 s2 = new Student4(222,"Aryan");
        //calling method to display the values of object
        s1.display();
        s2.display();
    }
}

```

## Output

```

111 Karan
222 Aryan

```

## ▼ Method overloading

Method Overloading allows different methods to have the same name, but different signatures where the signature can differ by the number of input parameters or type of input parameters, or a mixture of both.

Method overloading is also known as **Compile-time Polymorphism, Static Polymorphism, or Early binding** in Java. In Method overloading compared to parent argument, child argument will get the highest priority.

## Different Ways of Method Overloading in Java

### ▼ Changing the Number of Parameters.

Method overloading can be achieved by changing the number of parameters while passing to different methods.

```

// Java Program to Illustrate Method Overloading
// By Changing the Number of Parameters

// Importing required classes
import java.io.*;

// Class 1
// Helper class
class Product {

    // Method 1

```

```

// Multiplying two integer values
public int multiply(int a, int b)
{
    int prod = a * b;
    return prod;
}

// Method 2
// Multiplying three integer values
public int multiply(int a, int b, int c)
{
    int prod = a * b * c;
    return prod;
}
}

// Class 2
// Main class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Creating object of above class inside main()
        // method
        Product ob = new Product();

        // Calling method to Multiply 2 numbers
        int prod1 = ob.multiply(1, 2);

        // Printing Product of 2 numbers
        System.out.println(
            "Product of the two integer value :" + prod1);

        // Calling method to multiply 3 numbers
        int prod2 = ob.multiply(1, 2, 3);

        // Printing product of 3 numbers
        System.out.println(
            "Product of the three integer value :" + prod2);
    }
}

```

## ▼ Changing Data Types of the Arguments.

In many cases, methods can be considered Overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

```

// Java Program to Illustrate Method Overloading
// By Changing Data Types of the Parameters

// Importing required classes
import java.io.*;

// Class 1

```

```
// Helper class
class Product {

    // Multiplying three integer values
    public int Prod(int a, int b, int c)
    {

        int prod1 = a * b * c;
        return prod1;
    }

    // Multiplying three double values.
    public double Prod(double a, double b, double c)
    {

        double prod2 = a * b * c;
        return prod2;
    }
}

class GFG {
    public static void main(String[] args)
    {

        Product obj = new Product();

        int prod1 = obj.Prod(1, 2, 3);
        System.out.println("Product of the three integer value :" + prod1);
        double prod2 = obj.Prod(1.0, 2.0, 3.0);
        System.out.println("Product of the three double value :" + prod2);
    }
}
```

## ▼ Changing the Order of the Parameters of Methods

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods. For example, if the parameters of method 1 are (String name, int roll\_no) and the other method is (int roll\_no, String name) but both have the same name, then these 2 methods are considered to be overloaded with different sequences of parameters.

```
// Java Program to Illustrate Method Overloading
// By changing the Order of the Parameters

// Importing required classes
import java.io.*;

// Class 1
// Helper class
```

```

class Student {

    // Method 1
    public void StudentId(String name, int roll_no)
    {
        System.out.println("Name :" + name + " " + "Roll-No :" + roll_no);
    }
    // Method 2
    public void StudentId(int roll_no, String name)
    {
        // Again printing name and id of person
        System.out.println("Roll-No :" + roll_no + " " + "Name :" + name);
    }
}

// Class 2
// Main class
class GFG {

    public static void main(String[] args)
    {

        // Creating object of above class
        Student obj = new Student();

        // Passing name and id
        // Note: Reversing order
        obj.StudentId("Spyd3r", 1);
        obj.StudentId(2, "Kamlesh");
    }
}

```